

Shape Morphing in Geometry Nodes

Theory, standard algorithms, and a plan for `MorphNode`

BlenderMathAnim — `geometry_nodes/nodes.py`

Contents

1	The problem, concretely	1
2	The correspondence problem	2
2.1	Map both shapes to a common domain	2
2.2	Skip the mesh entirely	2
2.3	Discrete matching heuristics	2
3	The interpolation problem	3
4	What is feasible inside geometry nodes	3
5	Implemented plan	3
6	The supermesh pass	4
7	Roadmap	5

1 The problem, concretely

`MorphNode` currently computes, for every point p_i of the source geometry S_1 ,

$$x_i(t) = (1 - t)p_i + tq_i, \quad q_i = \text{position of point } i \text{ of } S_2, \quad (1)$$

with t the `Morph Parameter`. Two independent things are wrong with it, and they are worth separating because they have different fixes.

Vertex counts. The pairing $i \mapsto i$ is only defined when $N = |S_1| \leq M = |S_2|$. Blender’s `Sample Index` does *not* clamp an out-of-range index: it returns the zero vector, so every surplus source point is sent to the world origin. Measured on a 114-point UV sphere morphed into an 8-point cube: 106 points land on exactly $(0, 0, 0)$. In the picture-frame-into-arrow example of `MorphModifier` the frame carries $4 \times 32 = 128$ points and the arrow 100, so 28 points collapse into a knot — and because the arrow happens to stand *at* the origin, the knot hides at its foot and reads as part of the shape.

Edges and faces. Worse, and independent of the counts: the index of a point is its position in a storage array, not a property of the surface. Point 7 of a swept rectangle has nothing to do with point 7 of a cone. Formula (1) therefore transports the source’s connectivity along essentially random trajectories, and the surface folds through itself on the way. The target’s edges and faces never enter the computation at all: what comes out at $t = 1$ is the source’s mesh with the target’s point positions, which is only the target if the two meshes were compatible to begin with.

The literature calls these the *correspondence problem* and the *interpolation* (or *vertex path*) *problem*, and treats them separately [2, 11]. Everything below follows that split.

2 The correspondence problem

The goal is a bijection $\varphi : S_1 \rightarrow S_2$ between the two surfaces, together with a mesh that represents both — a *supermesh* whose connectivity is valid for the source *and* the target. Given that, the morph is a per-vertex interpolation and the edges and faces take care of themselves.

2.1 Map both shapes to a common domain

The dominant family. Both surfaces are parameterised over one domain, correspondence is composition through it, and the supermesh is the overlay of the two parameterisations.

- **Sphere as domain (genus 0).** Kent, Carlson and Parent [8] project two star-shaped polyhedra onto a common sphere, merge the two projected topologies into one, and interpolate. This is the original recipe and still the clearest one to implement.
- **Harmonic and barycentric maps.** For meshes with boundary, fix the boundary to a convex polygon and solve a linear system for the interior; Tutte’s theorem guarantees the embedding is valid [19], and Floater’s weights give a smooth, shape preserving version [6]. Kanai, Suzuki and Kimura use harmonic maps for metamorphosis of arbitrary triangular meshes [7].
- **Base domains and consistent parameterisation.** Cut both meshes into matching patches over a common coarse base mesh: multi resolution analysis of arbitrary meshes [5], consistent mesh parameterisations [12], cross-parameterisation and compatible remeshing [9], inter-surface mapping [13]. These handle higher genus and let the user pin feature correspondences (nose to nose, tip to tip).

2.2 Skip the mesh entirely

If the shapes are represented as functions rather than triangles, no correspondence is needed: interpolate the *representation*.

- **Implicit functions.** Blend the defining scalar fields and re-extract the surface [18].
- **Distance fields.** Interpolate signed distance fields, with a warp to keep features aligned [4].
- **Optimal transport.** Treat both shapes as measures and follow the Wasserstein geodesic; convolutional Wasserstein distances make this practical on geometric domains [16]. This is the principled answer to “which point goes where” when the two point sets differ in size, since transport plans are not required to be bijections.

Topology changes (a torus becoming a ball) come for free here and are essentially impossible in the mesh-based family.

2.3 Discrete matching heuristics

Cheap approximations that do not build a supermesh, which is the class the node can realistically use:

- **Rigid pre-alignment.** Centre both shapes (and optionally normalise scale and principal axes) before matching. This is the initialisation step of ICP [3]; without it every geometric matching criterion is dominated by where the two objects happen to sit.
- **Nearest-point projection.** Send each source point to the closest point of the target surface. Fast, order-independent, and surjective onto nothing in particular: it is many-to-one, so it is a shrink-wrap rather than a bijection. It respects geometry, which is exactly what index pairing does not.

- **Optimal assignment.** With equal counts, choose the permutation minimising $\sum_i \|p_i - q_{\sigma(i)}\|^2$ — the assignment problem, solvable in $O(n^3)$ by the Hungarian method [10]. This is the discrete cousin of optimal transport and removes the twisting that nearest-point matching leaves behind, at the price of a global solve.

3 The interpolation problem

Given corresponding points, formula (1) moves each along a straight line. That is the naive choice and it is visibly wrong for large deformations: a rotating limb shrinks towards the centre of rotation, because the straight line between two points of a rotating rigid body cuts the chord instead of following the arc. Standard alternatives:

- **Intrinsic interpolation (2D).** Interpolate edge lengths and turning angles instead of coordinates and reconstruct the polygon [14, 15].
- **As-rigid-as-possible interpolation.** Interpolate the affine map of each triangle by splitting it into rotation and stretch, and solve a least-squares problem for the vertex positions that best agrees with all of them [1]. The same machinery underlies ARAP surface modelling [17].
- **Gradient-domain / Poisson blending.** Interpolate the deformation gradients and integrate.

All three need a global linear solve per frame, which puts them out of reach of a pure geometry-node graph.

4 What is feasible inside geometry nodes

A geometry-node graph evaluates *fields*: one expression per element, with no ordering, no iteration to convergence, and no way to build new connectivity beyond what the fixed operators provide. That rules a lot out.

Method	Needs	In geometry nodes?
Index pairing	nothing	yes (today)
Index rescaling	two counts	yes
Centroid / scale alignment	attribute statistics	yes
Nearest-point projection	spatial query	yes (Geometry Proximity)
Curve arc-length pairing	resampling	yes (curves only)
Optimal assignment	global $O(n^3)$ solve	no
Spherical parameterisation	iterative relaxation	no (needs Python)
Supermesh construction	topology surgery	no (needs Python)
ARAP interpolation	linear solve per frame	no (needs Python)

The three rows marked “yes” that are not already there are what the plan below implements. The rest belong in a Python pre-pass that writes a compatible pair of meshes once, after which the existing node interpolates them correctly — see §7.

5 Implemented plan

Three steps, in the order the graph applies them. Each is a frame in the node group.

Step 1 — index rescaling (frame Index Pairing)

Replace $i \mapsto i$ with the monotone map

$$j(i) = \text{round}\left(i \cdot \frac{M-1}{N-1}\right), \quad N = |S_1|, M = |S_2|, \quad (2)$$

clamped to $[0, M-1]$, with $j = 0$ when $N \leq 1$. The point counts come from **Domain Size** on both inputs, so the map adapts to whatever is plugged in.

What it fixes: the index is never out of range, so nothing is ever sent to the origin — the count mismatch stops being a crash-like artefact and becomes a mere loss of resolution. When $N > M$ several source points share a destination; when $N < M$ the source samples the target evenly rather than only its first N points. *What it does not fix:* the pairing is still storage order.

Step 2 — centroid alignment (frame Alignment)

With

$$c_k = \frac{1}{|S_k|} \sum_{p \in S_k} p \quad (\text{Attribute Statistic, mean of Position}),$$

correspondence is computed between the *centred* shapes $S_1 - c_1$ and $S_2 - c_2$, and the destination is mapped back by $+c_2$.

Note that for a *linear* blend of an *index* pairing this is a no-op: $(1-t)(p - c_1) + t(q - c_2)$ plus the interpolated centroid $(1-t)c_1 + tc_2$ is exactly $(1-t)p + tq$. It matters only because Step 3 matches by distance, and distance is meaningless while the two shapes sit metres apart. It is the translation-only case of the rigid pre-registration that every geometric matcher assumes [3].

Step 3 — nearest-surface pairing (frame Nearest Pairing)

Pair by geometry instead of by index:

$$\varphi(p) = c_2 + \arg \min_{y \in S_2 - c_2} \|(p - c_1) - y\|, \quad (3)$$

evaluated by **Geometry Proximity** against the centred target, with **Target Element = Faces** so the destination may lie anywhere on the target surface rather than only on its vertices.

What it fixes: the destination of a point now depends on where the point *is*, not on when it was created. Neighbouring source points get neighbouring destinations, so the source’s edges and faces stay coherent instead of tangling, and the count question disappears entirely — a continuous surface has a nearest point for every source point whatever the two resolutions are. *What it does not fix:* the map is many-to-one. At $t = 1$ the source is shrink-wrapped onto the target: the silhouette is right and the surface lies on the target, but concavities the projection cannot see stay uncovered, and the source’s connectivity, not the target’s, is what survives.

A **Match Nearest** boolean switches between (2) and (3); the blend (1) is unchanged behind it.

6 The supermesh pass

Implemented in `geometry_nodes/supermesh.py`. Everything above is a compromise forced by the graph; this is the thing that actually solves the problem, by refusing to pair anything at all.

Both shapes are written over one spherical domain, following Kent, Carlson and Parent [8], and *the domain’s own triangulation becomes the shared connectivity*. Where they merge the two projected topologies into an exact overlay — every vertex of both, plus every edge-edge crossing — the pass resamples instead: an icosphere is tessellated once at a chosen subdivision level, and each of its directions d is traced onto both surfaces,

$$p_A(d) = c_A + t_A(d) d, \quad p_B(d) = c_B + t_B(d) d, \quad (4)$$

with t found by Möller–Trumbore intersection against every triangle, vectorised and chunked. The *farthest* hit is kept, so a ray leaving a cone that stands on a cylinder lands on the outside of the pair rather than on the seam between them.

The trade is explicit. The overlay is exact and its output resolution is a consequence of the inputs; resampling loses whatever falls between two directions, but cannot produce degenerate triangles, needs no spherical polygon clipping, and makes the resolution a parameter — 642 points at subdivision 3, 2562 at 4, whatever the inputs carry.

What comes out is two vertex arrays over one face array. Feeding them to `MorphNode` as `Geometry 1` and `Geometry 2` with `Match Nearest off` makes index pairing exactly right, because point i of the one *is* point i of the other; the node needs no change. Alternatively one mesh carries the other’s positions as a point attribute, and a `Named Attribute` node reads the destination with no sampling at all.

Assumptions, and how they are reported. Equation (4) assumes each shape is star-shaped about its centre c (the mean of its vertices unless the caller says otherwise) and of genus 0. Neither is checked and forbidden; both are measured and reported:

- directions that meet no triangle are counted as *misses* and fall back to the surface point lying most nearly that way, which keeps the map total;
- the Euler characteristic $V - E + F$ of each input is computed, since $\chi \neq 2$ means no spherical parameterisation exists at all.

A torus of revolution run against a cube reports 510 of 642 directions missed and $\chi = 0$, i.e. genus 1 — which is the honest answer for the picture frame of `MorphModifier`, and the reason that particular morph needs a base domain with a handle rather than a sphere [12, 9, 13].

Verified. A 24×12 UV sphere against a 20-sided cone gives 642 points and 1280 faces with no misses and $\chi = 2$ for both; the geometry the node evaluates agrees with $(1 - t)p_A + tp_B$ to 1.2×10^{-7} at $t = 0, \frac{1}{4}, \frac{1}{2}, \frac{3}{4}, 1$, with the face count constant throughout — which is what “the connectivity is the same at every value of the morph” means in practice.

7 Roadmap

1. **Arc-length pairing for curve-like shapes.** Convert both shapes to curves and `Resample Curve` to a common count. Both problems vanish at once: equal counts, and index order becomes arc length, which *is* geometric. Cheap, and enough for outline morphs (letters, silhouettes, the picture frame of `MorphModifier`, which the sphere cannot take).
2. **Higher genus and feature control.** Replace the sphere of §6 by a common base domain with the right handles, and let the caller pin correspondences (tip to tip) [12, 9, 13].
3. **Better trajectories.** With the supermesh in place, replace the straight line by an as-rigid-as-possible path [1]; a per-frame least-squares solve, so again Python.

References

- [1] M. Alexa, D. Cohen-Or, D. Levin. *As-rigid-as-possible shape interpolation*. SIGGRAPH, 2000.
- [2] M. Alexa. *Recent advances in mesh morphing*. Computer Graphics Forum 21(2), 2002.
- [3] P. J. Besl, N. D. McKay. *A method for registration of 3-D shapes*. IEEE Transactions on Pattern Analysis and Machine Intelligence 14(2), 1992.

- [4] D. Cohen-Or, D. Levin, A. Solomovici. *Three-dimensional distance field metamorphosis*. ACM Transactions on Graphics 17(2), 1998.
- [5] M. Eck, T. DeRose, T. Duchamp, H. Hoppe, M. Lounsbery, W. Stuetzle. *Multiresolution analysis of arbitrary meshes*. SIGGRAPH, 1995.
- [6] M. S. Floater. *Parametrization and smooth approximation of surface triangulations*. Computer Aided Geometric Design 14(3), 1997.
- [7] T. Kanai, H. Suzuki, F. Kimura. *Metamorphosis of arbitrary triangular meshes*. IEEE Computer Graphics and Applications 20(2), 2000.
- [8] J. R. Kent, W. E. Carlson, R. E. Parent. *Shape transformation for polyhedral objects*. SIGGRAPH, 1992.
- [9] V. Kraevoy, A. Sheffer. *Cross-parameterization and compatible remeshing of 3D models*. ACM Transactions on Graphics (SIGGRAPH), 2004.
- [10] H. W. Kuhn. *The Hungarian method for the assignment problem*. Naval Research Logistics Quarterly 2, 1955.
- [11] F. Lazarus, A. Verroust. *Three-dimensional metamorphosis: a survey*. The Visual Computer 14(8–9), 1998.
- [12] E. Praun, W. Sweldens, P. Schröder. *Consistent mesh parameterizations*. SIGGRAPH, 2001.
- [13] J. Schreiner, A. Asirvatham, E. Praun, H. Hoppe. *Inter-surface mapping*. ACM Transactions on Graphics (SIGGRAPH), 2004.
- [14] T. W. Sederberg, E. Greenwood. *A physically based approach to 2-D shape blending*. SIGGRAPH, 1992.
- [15] T. W. Sederberg, P. Gao, G. Wang, H. Mu. *2-D shape blending: an intrinsic solution to the vertex path problem*. SIGGRAPH, 1993.
- [16] J. Solomon, F. de Goes, G. Peyré, M. Cuturi, A. Butscher, A. Nguyen, T. Du, L. Guibas. *Convolutional Wasserstein distances: efficient optimal transportation on geometric domains*. ACM Transactions on Graphics (SIGGRAPH), 2015.
- [17] O. Sorkine, M. Alexa. *As-rigid-as-possible surface modeling*. Symposium on Geometry Processing, 2007.
- [18] G. Turk, J. F. O’Brien. *Shape transformation using variational implicit functions*. SIGGRAPH, 1999.
- [19] W. T. Tutte. *How to draw a graph*. Proceedings of the London Mathematical Society, 1963.